

1. 리눅스 기본 개념 (경로/홈 디렉토리)

리눅스/Unix 계열 시스템에서 모든 경로는 최상위인 `/ (root)`에서 시작합니다.

키워드	설명
<code>/</code>	모든 경로의 시작점(루트 디렉토리)
<code>~</code>	현재 사용자 홈 디렉토리(Home)
<code>.</code>	현재 디렉토리
<code>..</code>	현재 디렉토리의 상위 폴더

환경	홈 디렉토리 위치
mac/Linux	<code>/home/사용자명</code>
Windows (Git Bash)	<code>/c/Users/사용자명</code>

2. 파일/폴더 위치 및 탐색 명령어

(1) `pwd` - 현재 내가 있는 위치 확인

```
pwd
```

TIP: 길을 잃은 것 같을 때, 가장 먼저 확인

(2) `ls` - 현재 폴더 안에 무엇이 있는지 보기

```
ls
ls -l      # 권한/소유자/수정시간까지 자세히
ls -a      # 숨김파일까지 확인 (.git 등)
ls -al     # 가장 많이 사용되는 조합
```

상황	왜 필요한가
<code>.git</code> 폴더 확인할 때	Git이 활성화된 폴더인지 체크
파일 접근 권한 문제	<code>-l</code> 로 권한 확인 가능

(3) `cd` - 폴더 이동하기

```
cd 프로젝트폴더명
cd ..          # 상위 폴더로
cd ~           # 홈 디렉토리로 돌아가기
```

TIP: Tab 자동완성 사용하면 폴더명 전부 안 쳐도 됨

3. 파일/폴더 조작 명령어

(1) `mkdir` - 새 폴더 만들기

```
mkdir myapp
mkdir -p src/components # 중첩 폴더 한 번에 생성
```

(2) `touch` - 빈 파일 만들기

```
touch index.html
touch app.js README.md
```

(3) `rm` - 파일 삭제

```
rm test.txt
rm -r myapp # 폴더 삭제
rm -rf myapp # 폴더 강제 삭제
```

(4) `mv` - 이동 또는 이름 변경

```
mv a.txt b.txt # 이름 변경
mv a.txt backup/ # 다른 폴더로 이동
```

(5) `cp` - 복사

```
cp a.txt backup.txt # 파일 복사
cp -r src backup # 폴더 복사
```

4. 파일 내용 확인/읽기/검색

명령어	역할	대표 사용 사례
<code>cat</code>	파일 내용을 한 번에 전부 출력	짧은 파일 보기
<code>head</code>	위에서 일부만 보기	설정 파일 구조 훑어보기
<code>tail</code>	아래쪽 일부 보기	로그 확인
<code>tail -f</code>	실시간 로그	서버 log 모니터링
<code>grep</code>	내용 검색	특정 문자열 찾기

(1) `cat`

```
cat README.md
```

(2) `head`

```
head -n 10 config.json    # 처음 10줄만 보기
```

(3) tail

```
tail -n 20 logs/app.log    # 마지막 20줄 보기
```

(4) tail -f (실시간 로그)

```
tail -f logs/app.log
```

TIP: 서버 환경에서 가장 많이 사용하는 명령어 중 하나

(5) grep - 특정 텍스트 검색

```
grep "DB_HOST" .env
grep -n "ERROR" logs/app.log    # 줄번호까지
```

5. 시스템 정보 / 사용자 / 경로 확인 등 [참고]

명령어	설명
whoami	현재 로그인한 사용자 확인
which	명령어(실행파일)의 실제 위치 확인
du -sh	폴더나 파일 용량 확인
df -h	전체 디스크 사용량 확인
chmod	실행 권한 주기
chown	소유자 변경(관리자 권한)
tar	압축 / 압축 해제
curl	URL 내용을 가져와 출력하거나 저장
wget	URL에서 직접 다운로드

6. Git

Git은 "현재 상태"를 아래 3단계로 나누어 관리합니다.

단계	명령어	설명
작업 디렉토리	(변경 중)	실제 파일 수정 중
스테이징 영역	git add	커밋 후보 등록

단계	명령어	설명
저장소	<code>git commit</code>	버전 기록 생성

(1) 저장소 처음 만들기

```
git init
```

이 순간 `.git`이라는 숨김폴더 생성 → 여기 안에 버전 정보가 보관됨

(2) 원격 저장소 복제

```
git clone <url>
```

(3) 변경사항 올리기

```
git add 파일명      # 특정 파일 올리기
git add .           # 모든 변경사항 staging
git commit -m "메시지"
git push
```

(4) 최신 코드 가져오기

```
git pull
```

팀 프로젝트에서는 `push`보다 `pull`을 더 많이 쓰는 순간이 옵니다
(다른 사람이 작업했는지 확인해야 충돌 방지)

7. Git 복구 & 실수 되돌리기

상황	해결 명령
수정했는데 되돌리고 싶음	<code>git restore 파일명</code>
모든 파일 변경 취소	<code>git restore .</code>
staging 취소 (<code>git add</code> 되돌림)	<code>git restore --staged 파일명</code> / <code>git rm --cached 파일명</code>
커밋 취소(되돌리고 작업물은 유지)	<code>git reset --soft COMMIT_ID</code>
커밋/스테이징 취소 (작업물은 유지)	<code>git reset COMMIT_ID</code>
커밋/스테이징/작업 모두 완전 삭제	<code>git reset --hard COMMIT_ID</code> ⚠ 주의

8. .gitignore - Git에 포함시키지 않을 파일 관리

Git으로 버전 관리하면 안 되는 파일:

예시	이유
node_modules/	용량 큼 + 다시 설치 가능
.env	비밀번호/토큰 등 민감정보
로그(*.log)	기록용이며 버전 관리 X
IDE 설정(.vscode/)	사용자마다 다름

.gitignore 파일 예시:

```
node_modules/  
.env  
.vscode/  
*.log
```